

CS 219 Operating Systems

Spring 2025-26

Quiz 1 (20 marks)

A

Name: _____ Roll No.: _____

Q1. State TRUE or FALSE and justify. [7 x 1 = 7 marks] **No correct justification, no marks.**

- a. The pipe system call creates two file descriptors, fd[0] and fd[1]. Writes using fd[1] can be read using fd[0], and writes using fd[0] can be read using fd[1].

FALSE.

The pipe is an **unidirectional** channel for data transfer/communication and data written on fd[1] appears for read on fd[0].

- b. LDE (limited-direct execution) is not applicable to the *init* process — the first process the OS handcrafts during boot-up.

FALSE.

The init process is a user process, like any other user process, it is only different in the way it is created. Like any other user process it **cannot** directly execute privileged instructions to ensure correctness and isolation provided by OS services.

- c. The **exec** system call can be used only when the parent process forks — creates and starts a child process, not after a process has started executing.

FALSE

The exec system call is used to load and attach a program to a process. The call can be used at any point during execution of a process— just after process creation, at any point of its execution or just before it exits!

- d. Assuming a user process knows the PID of another process (from a pool of several READY processes), it has the capability to initiate a context switch and schedule the target process next on the CPU.

FALSE

Scheduling/assigning CPU to a process, is part of OS logic to share and multiplex a resource across execution instances (processes). While a process can potentially yield/give up its CPU, it cannot influence what process to execute next, that decision depends on the OS scheduling logic/policy.

- e. The PCB of a process resides in the process's data section of its memory.

FALSE

PCBs are data objects/variables of the OS program, store metadata information about the OS

abstractions and reside in the OS's (process) memory, along with its code (instructions).

- f. Two processes created and started simultaneously will **always** be alternatively scheduled one after the other by the operating system.

FALSE

CPU allocation to processes depends on the OS scheduling policy and the execution state of each process. Two processes may not be in the READY queue whenever a scheduling decision has to be made and/or the policy may choose sequences that do not alternate.

- g. A process waiting to be reaped can execute and process a pending signal.

FALSE

A process waiting to be reaped indicates the process has been marked implicitly or explicitly for termination, hence a re-execution of the process will violate semantics of reap. Also, waiting to be reaped, the only state the OS needs to maintain are some information in the PCB (notably the exit status), the rest of the code and data of the process can be cleared up and may not even exist for execution.

Q2.

- a. As part of the **signals** primitive available to processes, describe three main steps of the Operating System as part of the signal delivery and processing. **[3 marks]**

(i) Delivery of a signal

Using the pid of the target process, identify its PCB.

Check if the signal to be delivered is blocked by the process.

If not, queue the signal in the pending signal state of the PCB.

(ii) As part of target process scheduling on the CPU, check pending signal state in PCB.

If a signal is pending and is set to be ignored, ignore it.

(iii) If a signal is pending and is not ignored, and if signal handler is registered,

setup return state to jump signal handler when process is scheduled.

if not handler registered, perform default signal handler action via the OS.

b. List examples of two different types of signals and explain why they are different. [2 marks]

(i) SIGINT — interrupt a process

Can be blocked, ignored and handled via registration of a user function

(ii) SIGSTOP — pause a process

OS changes the execution state of a process (to BLOCKED) by updating the state variable in the PCB. Cannot be blocked, ignored or handled.

Q3. Consider the code given below. Assume that the code compiles with appropriate header files. [1 mark]

<pre>int main() { pid_t pid = fork(); write(1, "A", 1); wait(NULL); write(1, "B", 1); return 0; }</pre>	Is ABAB a valid output/display sequence? YES. Explain. Post the fork system call if the OS schedules the child process before the parent process, the output sequence is possible.
---	---

Q4. Which of these operations require privileged/non-privileged mode of the CPU? Give reasons. [2 x 1 = 2 marks]

a. Updating a register to indicate the system identifier as part of system call invocation.

Works/valid with non-privileged mode of execution of the CPU.

A user process uses the OS interface to invoke a system call and needs to also indicate which system call. A CPU register is a feasible location to store this information as CPU registers are maintained across privilege level switches.

b. Updating the execution STATE variable of a process's PCB.

Requires privileged mode of execution of the CPU.

PCBs are OS objects/variables and cannot be accessed directly from the user process and they contain information maintained by the OS (metadata) to manage its abstractions. The objects influence actions of the OS, e.g., change in execution state impact scheduling and cannot be handed over to user processes (which are users of OS services).

Q5. Given below is a process state diagram. For every numbered edge (edges marked 1, 2, 3, 4), state if the given transition is possible or not, and **explain**. [2 marks]

1. Invalid

All processes need to be reaped by the parent process before terminating.

2. Valid

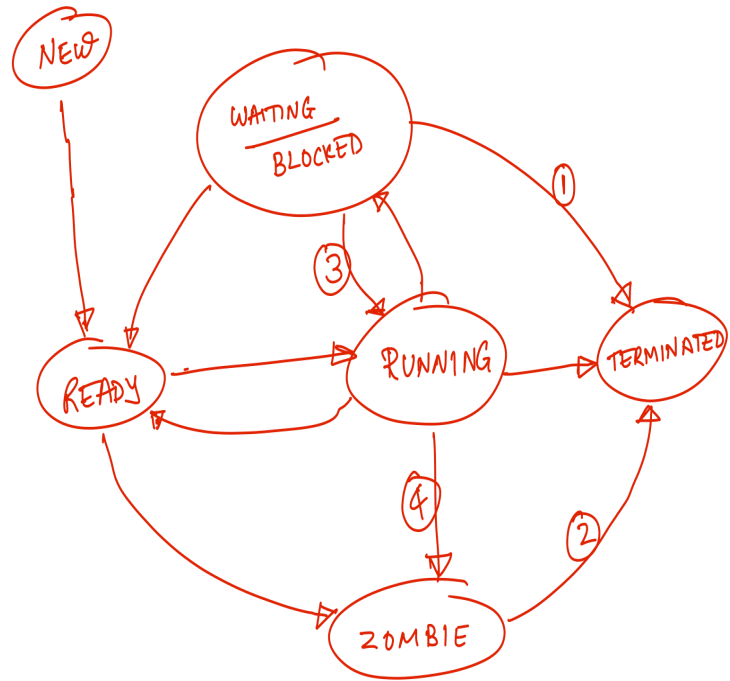
A process in ZOMBIE state is moved to TERMINATED state after reaping and all its metadata can be cleaned up

3. Invalid

All BLOCKED processes need to go past the scheduler to RUN and have to move to the READY state to indicate readiness to the scheduler

4. Valid

A process in running state can invoke the **exit** system call and directly move to ZOMBIE state for reaping.



Q6. Complete the following implementation of the **mycat** program, which intends to read the contents of the file and display it on the terminal. Assume that the code compiles with appropriate header files. Each blank has one semi-colon. **[2 marks]**

```
int main(int argc, char** argv) {
    int pid, fd;
    if(argc!=2) {
        fprintf(stdout, "usage: ./mycat filename\n");
        return -1;
    }

    pid=fork();
    if(pid==0) // child process
    {
        (i) close(STDIN_FILENO);

        (ii) fd = open(argv[1], O_RDWR);

        execv("/bin/cat", "/bin/cat"); // execute the cat program
    }
    else
        wait(NULL);
    return 0;
}
```

OR

(i) fd = open(argv[1], O_RDWR);

(ii) dup2(fd, STDIN_FILENO);

Q9. "OS is more relevant than ChatGPT!"

Your reaction, with explanation. **[1 mark]**

An introspective answer.

Rough work